Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Prof. Dr.-Ing. Sebastian Schlesinger
Fachbereich 2 - Duales Studium Wirtschaft & Technik

Übungsblatt 1

Digitale Technologien

Python Intro · Master Digitale Transformation

HWR Berlin, Wintersemester 2026/2027

Prof. Dr.-Ing. Sebastian Schlesinger

Lernziele: Eine Python-Arbeitsumgebung einrichten, kleine Programme entwickeln, Algorithmen nachvollziehen und Klassen sowie Polymorphie einsetzen.

Arbeitsweise: Aufgaben 1–6 bilden den Einstieg; 7–10 dienen der anschließenden Vertiefung und dem Selbstlernen. Planen Sie insgesamt etwa 6–8 Stunden ein, abhängig von Vorkenntnissen und Installationen. Das Begleitnotebook `Python_Intro.ipynb` (Abschnitte A–D und G) hilft bei der Bearbeitung. Insgesamt: **100 Übungspunkte**. Schreiben Sie eigene Lösungen und erläutern Sie verwendete Hilfen kurz.

Aufgabe 1 (Python und VS Code einrichten) (12 Punkte)

a) Installieren Sie Python 3.12 oder neuer, VS Code sowie die Microsoft-Erweiterungen *Python* und *Jupyter*. Prüfen Sie im Terminal die Python-Version: `python3 --version` (macOS/Linux) bzw. `py -3 --version` (Windows).

b) Öffnen Sie einen neuen Projektordner `python-intro` in VS Code. Erstellen Sie über *Python: Create Environment* eine virtuelle Umgebung `.venv` und wählen Sie sie über *Python: Select Interpreter* aus.

c) Erstellen Sie `hello.py`, das Ihren Namen und `Hallo Python!` ausgibt. Führen Sie die Datei aus. Geben Sie zusätzlich mit `sys.version` und `sys.executable` Version und Pfad des verwendeten Interpreters aus.

d) Setzen Sie einen Breakpoint vor einer Ausgabe und starten Sie den Debugger. Ändern Sie eine Variable und beobachten Sie ihren Wert. Erklären Sie in der `README.md` den Unterschied zwischen Editor, Interpreter und virtueller Umgebung.

Nachweis: `hello.py`, dokumentierte Python-Version, Interpreterpfad und kurze Erklärung.

Aufgabe 2 (Git und GitHub einrichten) (8 Punkte)

a) Erstellen Sie ein GitHub-Konto, installieren Sie Git und prüfen Sie `git --version`. Legen Sie auf GitHub ein Repository `python-intro` mit README an und klonen Sie es über VS Code (*Git: Clone*) in einen separaten Ordner. Öffnen Sie diesen als Projekt.

b) Übernehmen Sie `hello.py` und Ihre README-Inhalte. Erstellen Sie die virtuelle Umgebung im geklonten Projekt neu. Tragen Sie `.venv/`, `__pycache__/` und `.ipynb_checkpoints/` in `.gitignore` ein.

c) Konfigurieren Sie Ihren Git-Namen und Ihre Commit-E-Mail. Erstellen Sie einen Commit und pushen Sie ihn. Legen Sie einen Branch `begruessung` an, ändern Sie die Begrüßung und pushen Sie den Branch. Öffnen und mergen Sie einen Pull Request; aktualisieren Sie anschließend lokal `main`.

d) Erklären Sie Git vs. GitHub sowie Commit vs. Push. Halten Sie Repository-URL und Commit-Kennung in der README fest. Zugangsdaten gehören nicht ins Repository.

Hilfe: VS Code: Python-Tutorial; GitHub: Hello World.

Aufgabe 3 (Google Colab nutzen) (6 Punkte)

a) Melden Sie sich bei Google Colab an und erstellen Sie ein Notebook `blatt01.ipynb`. Fügen Sie eine Markdown-Zelle mit Titel und Lernziel sowie eine Codezelle mit `print("Hallo Colab!")` hinzu. Führen Sie diese aus.

b) Legen Sie in einer Zelle `preis = 19.90` und `menge = 3` fest und berechnen Sie in einer zweiten Zelle den Umsatz. Erklären Sie, warum die Ausführungsreihenfolge zählt.

c) Starten Sie die Laufzeit neu und führen Sie alle Zellen von oben nach unten aus. Laden Sie das Notebook als `.ipynb` herunter und nehmen Sie es in Ihr Git-Repository auf.

d) Nennen Sie je einen Vorteil der lokalen Arbeit und von Colab. Was müssen Sie sichern, bevor eine Colab-Laufzeit endet? Unterscheiden Sie Notebook und Dateien im Laufzeitverzeichnis.

Nachweis: Ausführbares Notebook und vier bis sechs Sätze in der README. Die weiteren Programmieraufgaben können Sie lokal oder in Colab lösen.

Aufgabe 4 (Ein Python-Programm mit Docker ausführen) (8 Punkte)

a) Installieren und starten Sie Docker Desktop gemäß der offiziellen Anleitung (unter Linux alternativ Docker Engine). Prüfen Sie im lokalen Terminal:

```
docker --version
docker run --rm hello-world
```

b) Erstellen Sie neben `hello.py` eine Datei `Dockerfile` mit folgendem Inhalt. Für dieses Programm sind keine zusätzlichen Python-Pakete erforderlich.

```
FROM python:3.12-slim
WORKDIR /app
COPY hello.py .
CMD ["python", "hello.py"]
```

c) Bauen und starten Sie das Image im Projektordner:

```
docker build -t python-intro .
docker run --rm python-intro
```

Vergleichen Sie die Ausgabe mit der lokalen Ausführung. Ändern Sie die Begrüßung in `hello.py`. Was wird ohne erneuten Build ausgegeben, was nach einem neuen Build?

d) Erklären Sie die Begriffe Dockerfile, Image und Container sowie den Unterschied zwischen einem Container und einer Python-Umgebung `.venv`.

Nachweis: Dockerfile, Befehle und beobachtete Ausgaben in der README. Falls die Installation auf Ihrem Gerät scheitert, dokumentieren Sie Fehlermeldung und Lösungsversuch; bearbeiten Sie die Python-Aufgaben zunächst in Colab weiter.

Ergebnisse organisieren: Speichern Sie Aufgaben 5–10 in `blatt01.py` oder im Notebook. Funktionen sollen Ergebnisse mit `return` zurückgeben; Ausgaben dienen nur der Darstellung. Ergänzen Sie die genannten Tests und mindestens einen selbst gewählten Randfall pro Funktion. Nutzen Sie nur die Python-Standardbibliothek.

Aufgabe 5 (Kleine Programme: Variablen, Bedingungen und Schleifen) (12 Punkte)

a) Temperaturumrechner (3 P.): Lesen Sie eine Celsius-Temperatur mit `input` ein, wandeln Sie sie mit `float` in eine Zahl um und berechnen Sie $F = C \cdot 9/5 + 32$. Geben Sie das Ergebnis mit einer Nachkommastelle aus. Prüfen Sie $0 \mapsto 32$, $100 \mapsto 212$ und $-40 \mapsto -40$. Warum ist eine Typumwandlung nötig?

b) Versandkosten (3 P.): Schreiben Sie `versand(warenwert)`. Für einen nichtnegativen Warenwert unter 50 Euro betragen die Versandkosten 4,90 Euro, ab 50 Euro sind sie null. Negative Werte sollen einen `ValueError` auslösen. Prüfen Sie 0, 49,99, 50 und −1.

c) FizzBuzz (3 P.): Geben Sie die Zahlen 1 bis einschließlich 30 aus. Ersetzen Sie durch 3 teilbare Zahlen durch `Fizz`, durch 5 teilbare durch `Buzz` und durch beide teilbare durch `FizzBuzz`. Nutzen Sie `%`. Warum ist die Reihenfolge der Bedingungen wichtig?

d) Umsätze (3 P.): Gegeben sind `preise = [12, 8, 15, 10]` und `mengen = [2, 5, 1, 3]`. Berechnen Sie mit einer Schleife die Einzelumsätze, den Gesamtumsatz und die Anzahl der Einzelumsätze ab 30 Euro. Erwartung: `[24, 40, 15, 30]`, Gesamtumsatz 109, Anzahl 2.

Aufgabe 6 (Funktionen, Strings und Dictionaries) (10 Punkte)

a) Durchschnitt (4 P.): Implementieren Sie `durchschnitt(werte)` für eine Liste von Zahlen ohne `sum`. Bei einer leeren Liste soll `ValueError` ausgelöst werden. Prüfen Sie:

```
assert durchschnitt([2, 4, 6]) == 4
assert durchschnitt([5]) == 5
assert durchschnitt([-4, 2]) == -1
```

Ergänzen Sie einen Test mit `try/except`, der den Fehlerfall überprüft.

b) Palindrom (3 P.): Implementieren Sie `ist_palindrom(text)`. Ignorieren Sie Groß-/Kleinschreibung und Leerzeichen; andere Zeichen bleiben erhalten. `LLagerregal"` und `Äb ba"` liefern `True`, `PPython"` liefert `False`. Der leere String gilt hier als Palindrom. Tipp: `lower`, `replace` und Slicing.

c) Häufigkeiten (3 P.): Zählen Sie in `["Web", SShop", "Web", PPartner", "Web"]` die Häufigkeit jedes Werts mit einem Dictionary und einer Schleife, ohne `Counter`. Erwartung: `{"Web: 3, SShop: 1, PPartner: 1}`. Verpacken Sie den Code in `haeufigkeiten(werte)`; für `[]` ist das Ergebnis `{}`. Tipp: `dict.get`.

Reflexion: Erklären Sie an einer Ihrer Funktionen den Unterschied zwischen `print` und `return`, zwischen Parameter und Argument sowie zwischen `=` und `==`.